
天津大学

智能搜索与问答

实验 nmt_experiment



学 院 智能与计算学部
专 业 计算机科学与技术
学 号 3022244006
姓 名 樊建琦

1. 实验相关背景与模型算法介绍

1.1. NMT

神经机器翻译 (NMT) 是一种基于神经网络的机器翻译方法, 它利用深度学习模型将源语言句子直接映射到目标语言句子。相比传统的例如 RNN 等的基于规则或统计的方法, NMT 在翻译质量上取得了显著的提升, 并逐渐成为机器翻译领域的主流方法之一。

1.1.1. 结构

神经机器翻译的结构较为简单和直接: 首先, 通过一个称为编码器 (Encoder) 的神经网络, 将源语言句子编码为一个稠密向量。接着, 通过一个称为解码器 (Decoder) 的神经网络, 从该向量解码出目标语言句子。这样的神经网络模型通常称为“编码器-解码器” (Encoder-Decoder) 架构。如 Fig. 1所示。

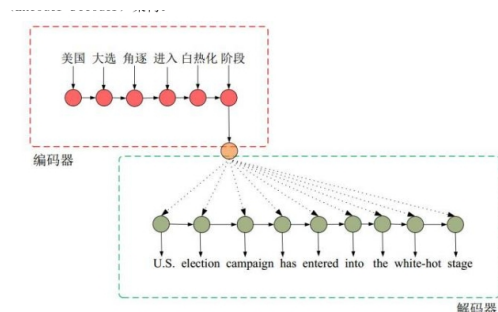


Figure 1. Encoder-Decoder

1.1.2. 常用模型——结合注意力机制

注意力机制主要对“编码器-解码器”结构进行了改进。普通的“编码器-解码器”结构采用前向循环神经网络编码器对源语言句子进行编码并将末尾的隐式状态作为上下文向量。而在基于注意力机制的“编码器-解码器”结构采用双向循环神经网络编码器对源语言句子进行编码。解码步骤中, 模型通过注意力机制选择性地关注源语言句子的不同部分, 动态地构建上下文向量。

1.1.3. 缺陷

词语表规模受限问题在 NMT 的实践中, 我们通常采用适当规模的词汇表来控制模型的时空开销, 一般词汇表规模介于 3 万至 5 万之间。对于词汇表未覆盖的词语, 模型只能将其替换为源语言端和目标语言端对应的 UNK 字符。由于词汇表相对有限, UNK 字符的使用不仅会影响模型对源语言句子语义信息的完整捕获, 也会影响用户理解模型生成的目标语言句子。这一问题在词形丰富的语言 (如德语) 中尤为突出。源语言翻译覆盖问题在神经机器翻译的解码过程中, 我们刚刚提到常用方法是通过注意力机制实现对源语言句子片段的自动调整, 以选择产生对应目标语言单词所需的关注度。然而, 由于注意力机制缺乏约束, 无法确保对源语言句子中的词语进行精准的关注, 因而可能出现“过翻译”和“欠翻译” (类比过拟合和欠拟合) 的现象。“过翻译”指源语言中不应重复翻译的词语被重复翻译, “欠翻译”则指应该被翻译但未被翻译的源语言词语。翻译不忠实问题对词语使用连续表示的方式, 一方面确实增强了神经机器翻译的泛化能力, 但同时另一方

面也增加了产生不忠实翻译的风险。不忠实的翻译指的是模型生成的目标语言词语可以确保目标语言句子的流畅性, 但无法准确地传达源语言句子的语义信息。

1.2. Beam Search

Beam Search 与我们大一在算法课上学过的贪心算法很类似。使用贪心搜索来解码, 需要每一个时间步都取出一个条件概率最大的输出。而使用 Beam Search 解码, 在每一个时间步, 不再只保留当前分数最高的 1 个输出, 而是保留 num_beams 个。注意, 当 num_beams=1 时集束搜索就退化成了贪心搜索。Beam Search 示意图如 Fig. 4:

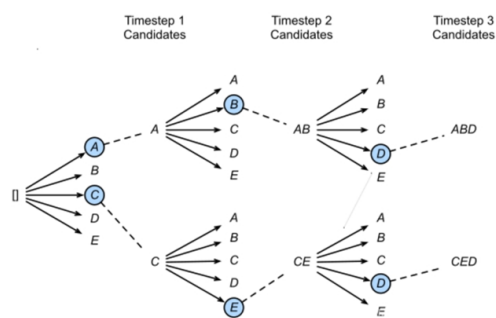


Figure 2. Beam Search

这张图中, num_beam=2。在第一个时间步, 选取最优的两个 A 和 C, 其他三个抛弃了; 第二步会基于这两个结果继续进行生成, 在 A 这个分支可以得到 5 个候选, [AA],[AB],[AC],[AD],[AE], C 也得到 5 个, 需要注意的是此时会对这 10 个进行统一排名再保留最优的两个, 而非将 A 和 C 的候选各自排名; 第三步同理, 也会从新的 10 个候选人里再保留最好的两个, 最后得到了 [ABD],[CED] 两个结果。可以发现, beam search 是一种牺牲时间换性能的方法。

1.3. BLEU 评分

本次实验我们需要用 BLEU 对模型的翻译结果进行评分。BLEU (Bilingual Evaluation Understudy) 评分是一种用于评估机器翻译和自然语言生成模型输出的质量的指标。基于双语词典和语言学特征, 通过计算翻译文本中与参考文本匹配的单词、词组和句子的准确率, 来评价机器翻译系统的性能。它具有简单、可比较和易于理解等优点, 为不同系统之间的性能比较提供了公平的基础。但需要注意的是, BLEU 也有一些使用的局限性。首先 BLEU 评分主要关注 n-gram 匹配, 可能忽略了句子的整体流畅度和语法正确性。同时需要注意, BLEU 需要高质量的参考文本, 且数量越多越好, 否则可能导致评分不准确。

2. 算法与代码分析

2.1. 训练 BPE 分词模型

```
1 #!/bin/bash
2
```

```

3 project_dir=/root/autodl-tmp/nlp_lab3/nmt_experiment
4
5 spm_train \
6   --input ${project_dir}/data/raw_corpus/train.zh-en.
   ↪ zh \
7   --model_prefix ${project_dir}/data/spm_data/spm.zh \
8   --vocab_size 8000 \
9   --character_coverage 1.0 \
10  --model_type bpe
11
12 spm_train \
13   --input ${project_dir}/data/raw_corpus/train.zh-en.
   ↪ en \
14   --model_prefix ${project_dir}/data/spm_data/spm.en \
15   --vocab_size 8000 \
16   --character_coverage 1.0 \
17   --model_type bpe

```

Code 1. 训练 BPE 分词模型

- 设置了变量 project_dir, 指定了项目的目录路径。
- 使用 mkdir -p 命令创建了目录
- 使用 spm_train 命令两次, 分别针对中文和英文文本数据进行训练。每次训练都指
- Input (指定输入文件的路径)、model_prefix (指定模型文件的前缀)、vocab_size (指定词汇表的大小, 这里设置为 8000)、character_coverage (指定字符覆盖率, 这里设置为 1.0, 表示覆盖所有字符) 和 model_type (指定模型类型, 这里设置为 BPE)

2.2. 实现 BPE 分词

这部分主要由 spm_encode.sh 完成

```

1 #!/bin/bash
2
3 project_dir=/root/autodl-tmp/nlp_lab3/nmt_experiment
4 spm_corpus_dir=${project_dir}/data/spm_corpus
5 mkdir -p ${spm_corpus_dir}
6
7 for lang in "zh" "en"; do
8   for file_type in "train" "valid" "test"; do
9     spm_encode \
10      --model ${project_dir}/data/spm_data/spm.${lang}
   ↪ ${lang}.model \
11      --output_format piece \
12      --input ${project_dir}/data/raw_corpus/${file_type}.zh-en.${lang} \
13      --output ${spm_corpus_dir}/${file_type}.bpe.
   ↪ zh-en.${lang}
14
15     echo "${file_type}.zh-en.${lang} complete"
16   done
17 done

```

Code 2. 实现 BPE 分词

- 指定项目目录 project_dir 和分词后的语料库输出目录 spm_corpus_dir。
- 使用 mkdir -p 命令创建了分词后的语料库输出目录。

- 定义包含语言对的数组 langs, 其中包括了中文和英文。
- 使用 for 循环遍历语言对数组, 并对每种语言的文本数据进行分词处理。
- 在循环中使用 spm_encode 命令对文本进行分词处理, 输出分词结果到指定目录。
- 循环结束时输出处理完成的消息。

2.3. 生成词表

主要通过 build_vocab.sh 调用 build_vocab.py 实现。

```

1 #!/bin/bash
2
3 project_dir=/root/autodl-tmp/nlp_lab3/nmt_experiment
4 python -u ${project_dir}/nmt/build_vocab.py \
5   --corpus-path ${project_dir}/data/spm_corpus/train.
   ↪ bpe.zh-en.zh \
6   --vocab-path ${project_dir}/data/spm_corpus/vocab.zh
   ↪ .json \
7
8 python -u ${project_dir}/nmt/build_vocab.py \
9   --corpus-path ${project_dir}/data/spm_corpus/train.
   ↪ bpe.zh-en.en \
10  --vocab-path ${project_dir}/data/spm_corpus/vocab.en
   ↪ .json \

```

Code 3. build_vocab.sh

build_vocab(corpus_path: str, vocab_path: str) 用于构建词汇表。它接受两个参数, corpus_path 是语料库文件的路径, vocab_path 是要保存词汇表的文件路径。该函数首先创建一个包含三个特殊标记的空的词汇表 vocab, 然后打开语料库文件, 逐行读取每个句子, 将句子分割成单词, 并将单词添加到词汇表中。最后, 将构建好的词汇表保存到指定的文件中。cli_main() 是命令行接口的入口函数。它首先创建了一个 ArgumentParser 对象, 然后添加了两个参数, -corpus-path 和 -vocab-path, 分别用于指定语料库文件的路径和词汇表文件的路径。接着使用 parser.parse_args() 方法解析命令行参数, 并将结果存储在 args 中。最后, 调用 build_vocab 函数, 传入语料库文件路径和词汇表文件路径, 完成词汇表的构建。

2.4. 去除长句子

```

1 #!/bin/bash
2
3 project_dir=/root/autodl-tmp/nlp_lab3/nmt_experiment
4 python -u ${project_dir}/nmt/length_filter.py \
5   --src-path ${project_dir}/data/spm_corpus/train.bpe.
   ↪ zh-en.zh \
6   --tgt-path ${project_dir}/data/spm_corpus/train.bpe.
   ↪ zh-en.en \
7   --max-src-length 200 \
8   --max-tgt-length 200 \
9   --max-length-ratio 1000 \
10  --output-src-path ${project_dir}/data/spm_corpus/
   ↪ train.bpe.filter.zh-en.zh \
11  --output-tgt-path ${project_dir}/data/spm_corpus/
   ↪ train.bpe.filter.zh-en.en
12

```

```

13 python -u ${project_dir}/nmt/length_filter.py \
14 --src-path ${project_dir}/data/spm_corpus/valid.bpe.
   ↪ zh-en.zh \
15 --tgt-path ${project_dir}/data/spm_corpus/valid.bpe.
   ↪ zh-en.en \
16 --max-src-length 200 \
17 --max-tgt-length 200 \
18 --max-length-ratio 1000 \
19 --output-src-path ${project_dir}/data/spm_corpus/
   ↪ valid.bpe.filter.zh-en.zh \
20 --output-tgt-path ${project_dir}/data/spm_corpus/
   ↪ valid.bpe.filter.zh-en.en
21
22 python -u ${project_dir}/nmt/length_filter.py \
23 --src-path ${project_dir}/data/spm_corpus/test.bpe.
   ↪ zh-en.zh \
24 --tgt-path ${project_dir}/data/spm_corpus/test.bpe.
   ↪ zh-en.en \
25 --max-src-length 200 \
26 --max-tgt-length 200 \
27 --max-length-ratio 1000 \
28 --output-src-path ${project_dir}/data/spm_corpus/
   ↪ test.bpe.filter.zh-en.zh \
29 --output-tgt-path ${project_dir}/data/spm_corpus/
   ↪ test.bpe.filter.zh-en.en

```

Code 4. length_filter.sh

主要使用 length_filter.sh 调用 length_filter.py 进行去除长句子的步骤。

```

1 import argparse
2
3 from argparse import Namespace
4
5
6 def length_filter(args: Namespace):
7     min_max_filter_count = 0
8     ratio_filter_count = 0
9     with open(args.src_path, "r", encoding="utf-8") as
   ↪ src_in, \
10         open(args.tgt_path, "r", encoding="utf-8") as
   ↪ tgt_in, \
11             open(args.output_src_path, "w", encoding="
   ↪ utf-8") as src_out, \
12                 open(args.output_tgt_path, "w", encoding
   ↪ ="utf-8") as tgt_out:
13         for src_line, tgt_line in zip(src_in
   ↪ , tgt_in):
14             src_line = src_line.strip()
15             tgt_line = tgt_line.strip()
16
17             src_length = len(src_line.split
   ↪ ())
18             tgt_length = len(tgt_line.split
   ↪ ())
19
20             if not (args.min_src_length <=
   ↪ src_length <= args.max_src_length):
21                 min_max_filter_count += 1
22                 continue
23

```

```

24         if not (args.min_tgt_length <=
   ↪ tgt_length <= args.max_tgt_length):
25             min_max_filter_count += 1
26             continue
27
28         if src_length / tgt_length >
   ↪ args.max_length_ratio or tgt_length / src_length
   ↪ > args.max_length_ratio:
29             ratio_filter_count += 1
30             continue
31
32         src_out.write(src_line + "\n")
33         tgt_out.write(tgt_line + "\n")
34
35     print(f"min_max_filter_count: {min_max_filter_count}
   ↪ ")
36     print(f"ratio_filter_count: {ratio_filter_count}")
37
38
39 def cli_main():
40     parser = argparse.ArgumentParser()
41     parser.add_argument("--src-path", required=True,
   ↪ type=str)
42     parser.add_argument("--tgt-path", required=True,
   ↪ type=str)
43
44     parser.add_argument("--min-src-length", default=1,
   ↪ type=int)
45     parser.add_argument("--max-src-length", default=1000
   ↪ , type=int)
46
47     parser.add_argument("--min-tgt-length", default=1,
   ↪ type=int)
48     parser.add_argument("--max-tgt-length", default=1000
   ↪ , type=int)
49
50     parser.add_argument("--max-length-ratio", default=3.
   ↪ 0, type=float)
51
52     parser.add_argument("--output-src-path", required=
   ↪ True, type=str)
53     parser.add_argument("--output-tgt-path", required=
   ↪ True, type=str)
54
55     args = parser.parse_args()
56     length_filter(args)
57
58
59 if __name__ == "__main__":
60     cli_main()

```

Code 5. length_filter.py

- min_max_filter_count 和 ratio_filter_count 初始化为 0, 用于统计被最小/最大长度限制或长度比例限制过滤掉的本对数量。
- 打开输入的源语言文件 (args.src_path) 和目标语言文件 (args.tgt_path), 以及输出的源语言文件 (args.output_src_path) 和目标语言文件 (args.output_tgt_path)。这里使用了 with 语句, 确保文件在使用完毕后自动关闭。

- 使用 `zip` 函数逐行读取源语言和目标语言的对应行，并在循环中对它们进行处理。
- 对于每一对源语言和目标语言的句子使用 `strip()` 方法去除首尾空白字符。然后分别计算源语言句子和目标语言句子的长度，再通过 `split()` 方法分割成单词并计算列表长度。
- 检查源语言和目标语言的长度是否在指定的最小和最大长度范围内。如果不在范围内，则将对应的计数器增加 1，并跳过当前文本对。
- 检查源语言和目标语言的长度比例是否超过了指定的最大长度比例 (`args.max_length_ratio`)，如果超过了，则将对应的计数器增加 1，并跳过当前文本对。

2.5. 具体实验流程介绍

```

1 import os
2 import os.path as osp
3 import sys
4 import tqdm
5 import json
6 import torch
7 import random
8 import logging
9 import argparse
10 import numpy as np
11 import torch.nn.functional as F
12 import sys
13 sys.path.append(osp.dirname(osp.dirname(osp.abspath(
14     ↪ __file__))))
15 from nmt.config import NMTConfig
16 from nmt.transformer import (
17     Encoder,
18     Decoder,
19     Transformer
20 )
21 # 添加sacrebleu导入
22 import sacrebleu
23 import heapq
24
25 logging.basicConfig(
26     format="%(asctime)s | %(levelname)s | %(name)s | %(
27     ↪ message)s",
28     datefmt="%Y-%m-%d %H:%M:%S",
29     level=os.environ.get("LOGLEVEL", "INFO").upper(),
30     stream=sys.stdout,
31 )
32 logger = logging.getLogger("nmt.train")
33
34 def load_vocab(path):
35     with open(path, mode="r", encoding="utf-8") as fin:
36         vocab = json.load(fin)
37     return vocab
38
39 def load_data(corpus_path, vocab):
40     data = []
41     with open(corpus_path, mode="r", encoding="utf-8")

```

```

45     ↪ as fin:
46         for line in fin:
47             line = line.strip()
48
49             tokens = line.strip().split()
50
51             token_ids = []
52             for token in tokens:
53                 # 2 is UNK
54                 token_id = vocab.get(token, 2)
55                 token_ids.append(token_id)
56
57             token_ids = torch.tensor(token_ids, dtype=
58     ↪ torch.long)
59             data.append(token_ids)
60
61     return data
62
63 class MTDataset(torch.utils.data.Dataset):
64     def __init__(self, src_corpus_path, tgt_corpus_path,
65     ↪ src_vocab, tgt_vocab):
66         self.src_data = load_data(src_corpus_path,
67     ↪ src_vocab)
68         self.tgt_data = load_data(tgt_corpus_path,
69     ↪ tgt_vocab)
70
71         assert len(self.src_data) == len(self.tgt_data)
72         self.eos_token = torch.tensor([0])
73
74     def __getitem__(self, index):
75         src_input = self.src_data[index]
76
77         src_input = torch.cat([src_input, self.eos_token
78     ↪ ])
79
80         tgt_input = self.tgt_data[index]
81
82         target = torch.cat([tgt_input, self.eos_token])
83
84         tgt_input = torch.cat([self.eos_token, tgt_input
85     ↪ ])
86
87         return {
88             "src_input": src_input,
89             "tgt_input": tgt_input,
90             "target": target
91         }
92
93     def __len__(self):
94         return len(self.src_data)
95
96     def padding_batch(values):
97         size = max(v.shape[0] for v in values)
98
99         batch_size = len(values)
100
101         # 1 is padding
102         res = values[0].new(batch_size, size).fill_(1)

```

```

100
101     for i, v in enumerate(values):
102         res[i, : len(v)].copy_(v)
103
104     return res
105
106
107 def collate_fn(batch):
108     src_input = padding_batch(
109         [item["src_input"] for item in batch]
110     )
111     tgt_input = padding_batch(
112         [item["tgt_input"] for item in batch]
113     )
114     target = padding_batch(
115         [item["target"] for item in batch]
116     )
117     return {
118         "src_input": src_input,
119         "tgt_input": tgt_input,
120         "target": target
121     }
122
123
124 def label_smoothed_nll_loss(lprobs, target, epsilon,
125                             ↪ padding_idx=None):
126     if target.dim() == lprobs.dim() - 1:
127         target = target.unsqueeze(-1)
128
129     nll_loss = -lprobs.gather(dim=-1, index=target)
130     smooth_loss = -lprobs.sum(dim=-1, keepdim=True)
131
132     if padding_idx is not None:
133         pad_mask = target.eq(padding_idx)
134         nll_loss.masked_fill_(pad_mask, 0.0)
135         smooth_loss.masked_fill_(pad_mask, 0.0)
136     else:
137         nll_loss = nll_loss.squeeze(-1)
138         smooth_loss = smooth_loss.squeeze(-1)
139
140     nll_loss = nll_loss.sum()
141     smooth_loss = smooth_loss.sum()
142     eps_i = epsilon / (lprobs.size(-1) - 1)
143     loss = (1.0 - epsilon - eps_i) * nll_loss + eps_i *
144         ↪ smooth_loss
145     return loss, nll_loss
146
147
148 def setup_seed(seed: int):
149     torch.manual_seed(seed)
150     torch.cuda.manual_seed(seed)
151     torch.cuda.manual_seed_all(seed)
152     np.random.seed(seed)
153     random.seed(seed)
154
155
156 def train(config_path):
157     config = NMTConfig.load_config(config_path)
158
159     setup_seed(config.seed)
160
161     src_vocab = load_vocab(config.src_vocab_path)

```

```

160     tgt_vocab = load_vocab(config.tgt_vocab_path)
161
162     logger.info("Source language vocab size: {}".format(
163         ↪ len(src_vocab)))
164     logger.info("Target language vocab size: {}".format(
165         ↪ len(tgt_vocab)))
166
167
168 train_data = MTDataset(
169     config.src_train_corpus_path,
170     config.tgt_train_corpus_path,
171     src_vocab,
172     tgt_vocab
173 )
174
175 valid_data = MTDataset(
176     config.src_valid_corpus_path,
177     config.tgt_valid_corpus_path,
178     src_vocab,
179     tgt_vocab
180 )
181
182 test_data = MTDataset(
183     config.src_test_corpus_path,
184     config.tgt_test_corpus_path,
185     src_vocab,
186     tgt_vocab
187 )
188
189 encoder = Encoder(
190     d_model=config.d_model,
191     num_layers=config.num_encoder_layers,
192     num_heads=config.num_heads,
193     d_ff=config.d_ff,
194     dropout=config.dropout,
195     vocab_size=len(src_vocab)
196 )
197
198 decoder = Decoder(
199     d_model=config.d_model,
200     num_layers=config.num_decoder_layers,
201     num_heads=config.num_heads,
202     d_ff=config.d_ff,
203     dropout=config.dropout,
204     vocab_size=len(tgt_vocab)
205 )
206
207 model = Transformer(encoder, decoder)
208 model.train()
209
210 device = torch.device("cuda") if config.cuda else
211     ↪ torch.device("cpu")
212 model = model.to(device)
213
214 optimizer = torch.optim.Adam(
215     model.parameters(),
216     config.learning_rate,
217     betas=(config.adam_beta1, config.adam_beta2)
218 )
219
220 train_dataloader = torch.utils.data.DataLoader(
221     train_data,
222     config.batch_size,

```

```

219         shuffle=True,
220         collate_fn=collate_fn
221     )
222     valid_dataloader = torch.utils.data.DataLoader(
223         valid_data,
224         config.batch_size,
225         shuffle=False,
226         collate_fn=collate_fn
227     )
228     test_dataloader = torch.utils.data.DataLoader(
229         test_data,
230         config.batch_size,
231         shuffle=False,
232         collate_fn=collate_fn
233     )
234
235     '''
236     for sample in test_dataloader:
237         src_input = sample["src_input"]
238         tgt_input = sample["tgt_input"]
239         target = sample["target"]
240     '''
241
242     best_val_nll_loss = 999.00;
243     for i in tqdm.tqdm(range(config.epoch)):
244
245         for step, sample in enumerate(train_dataloader):
246
247             src_input = sample["src_input"]
248             tgt_input = sample["tgt_input"]
249             target = sample["target"]
250
251             src_input = src_input.to(device)
252             tgt_input = tgt_input.to(device)
253             target = target.to(device)
254
255             output = model(
256                 src_input=src_input,
257                 tgt_input=tgt_input,
258             )
259
260             lprobs = F.log_softmax(output, dim=-1)
261
262             lprobs = lprobs.view(-1, lprobs.shape[-1])
263             target = target.view(-1)
264
265             loss, nll_loss = label_smoothed_nll_loss(
266                 ↪ lprobs, target, config.label_smoothing, 1)
267
268             num_tokens = target.ne(1).long().sum()
269
270             loss = loss / num_tokens
271             nll_loss = nll_loss / num_tokens
272
273             loss.backward()
274             optimizer.step()
275             optimizer.zero_grad()
276
277             # if (step + 1) % config.log_interval == 0:
278             #     logger.info(f"step: {step}, loss: {
279             ↪ loss.item(): .6f}, nll_loss: {nll_loss.item(): .6
280             ↪ f}")

```

```

278
279     # 每10轮在验证集上评估一次
280     if (i + 1) % 10 == 0 or i == 0:
281         valid_loss = 0.0
282         vali_nll_loss = 0.0
283         tokens_count = 0
284         for sample in valid_dataloader:
285
286             src_input = sample["src_input"]
287             tgt_input = sample["tgt_input"]
288             target = sample["target"]
289             src_input = src_input.to(device)
290             tgt_input = tgt_input.to(device)
291             target = target.to(device)
292
293             with torch.no_grad():
294                 output = model(
295                     src_input=src_input,
296                     tgt_input=tgt_input,
297                 )
298
299                 lprobs = F.log_softmax(output, dim=-
300                 ↪ 1)
301
302                 lprobs = lprobs.view(-1, lprobs.
303                 ↪ shape[-1])
304                 target = target.view(-1)
305
306                 loss, nll_loss =
307                 ↪ label_smoothed_nll_loss(lprobs, target, config.
308                 ↪ label_smoothing, 1)
309
310                 num_tokens = target.ne(1).long().sum
311                 ↪ ()
312
313                 valid_loss += loss.item()
314                 vali_nll_loss += nll_loss.item()
315                 tokens_count += num_tokens
316
317                 valid_loss /= tokens_count
318                 vali_nll_loss /= tokens_count
319                 logger.info(f"Epoch {i+1}/{config.epoch},
320                 ↪ valid_loss: {valid_loss: .6f}, vali_nll_loss: {
321                 ↪ vali_nll_loss: .6f}")
322
323                 if (vali_nll_loss < best_val_nll_loss):
324                     best_val_nll_loss = vali_nll_loss
325                     logger.info(f"Epoch {i+1} SOTA in val,
326                     ↪ performing nll as {best_val_nll_loss: .6f}")
327                     test(test_dataloader, device, model, config
328                     ↪ )
329
330                     torch.save(model.state_dict(), os.path.
331                     ↪ join(config.save_dir, f"best_checkpoint.pt"))
332
333                     # 添加BLEU评估
334                     translate_and_evaluate_bleu(model,
335                     ↪ test_dataloader, device, config, tgt_vocab)
336                     else:
337                         logger.info(f"Epoch {i+1}/{config.epoch}
338                         ↪ completed")
339
340     def test(test_dataloader, device, model, config):

```

```

328 test_loss = 0.0
329 test_nll_loss = 0.0
330 tokens_count = 0
331 for sample in test_dataloader:
332     src_input = sample["src_input"]
333     tgt_input = sample["tgt_input"]
334     target = sample["target"]
335
336
337     src_input = src_input.to(device)
338     tgt_input = tgt_input.to(device)
339     target = target.to(device)
340
341     with torch.no_grad():
342         output = model(
343             src_input=src_input,
344             tgt_input=tgt_input,
345         )
346
347         lprobs = F.log_softmax(output, dim=-1)
348
349         lprobs = lprobs.view(-1, lprobs.shape[-1])
350         target = target.view(-1)
351
352         loss, nll_loss = label_smoothed_nll_loss(
353             ↪ lprobs, target, config.label_smoothing, 1)
354
355         num_tokens = target.ne(1).long().sum()
356
357         test_loss = test_loss + loss.item()
358         test_nll_loss = test_nll_loss + nll_loss.
359         ↪ item()
360         tokens_count = tokens_count + num_tokens
361
362     test_loss = test_loss / tokens_count
363     test_nll_loss = test_nll_loss / tokens_count
364     logger.info(f"test_loss: {test_loss: .6f},
365         ↪ test_nll_loss: {test_nll_loss: .6f}")
366
367 # 新增函数: Beam Search解码实现
368 def beam_search_translate(model, src_input, encoder_out,
369     ↪ device, beam_size=5, max_len=100, eos_id=0):
370     """
371     使用Beam Search算法进行解码翻译
372
373     Args:
374         model: Transformer模型
375         src_input: 源语言输入
376         encoder_out: 编码器输出
377         device: 计算设备
378         beam_size: beam大小
379         max_len: 最大生成长度
380         eos_id: 结束标记ID
381
382     Returns:
383         最终的翻译序列
384     """
385     batch_size = src_input.size(0)
386     vocab_size = model.decoder.linear.out_features
387
388     # 存储每个batch样本的beam结果
389     batch_results = []

```

```

386
387 # 对batch中的每个样本单独处理
388 for batch_idx in range(batch_size):
389     # 提取当前样本的编码器输出和padding mask
390     single_encoder_out = encoder_out[0][:, batch_idx
391     ↪ :batch_idx+1]
392     single_encoder_padding_mask = encoder_out[1][
393     ↪ batch_idx:batch_idx+1]
394
395     # 初始化队列, 每个元素为(累积log概率, 当前序列, 已
396     ↪ 完成标记)
397     beam_queue = [(0.0, torch.LongTensor([[eos_id]]),
398     ↪ .to(device), False)]
399
400     # 已完成的序列
401     finished_beams = []
402
403     # beam search主循环
404     for step in range(max_len):
405         # 临时存储当前步骤的候选项
406         candidates = []
407
408         # 处理所有当前beam中的候选项
409         for cum_log_prob, seq, is_finished in
410             ↪ beam_queue:
411             # 如果序列已完成, 将其添加到候选项, 不再扩
412             ↪ 展
413             if is_finished:
414                 candidates.append((cum_log_prob, seq
415                 ↪ , is_finished))
416                 continue
417
418         # 用于当前序列的解码器输入
419         decoder_input = seq
420
421         # 解码器前向传播
422         with torch.no_grad():
423             decoder_out = model.decoder(
424                 ↪ decoder_input, single_encoder_out,
425                 ↪ single_encoder_padding_mask)
426
427         # 获取最后位置的log概率
428         log_probs = F.log_softmax(
429             ↪ decoder_out[:, -1, :], dim=-1)
430
431         # 获取前beam_size个最大概率的词
432         topk_log_probs, topk_indices =
433             ↪ log_probs.topk(beam_size)
434
435         # 将新候选项添加到列表
436         for i in range(beam_size):
437             token_id = topk_indices[0, i].
438             ↪ item()
439             log_prob = topk_log_probs[0, i].
440             ↪ item()
441
442         # 计算新的累积概率
443         new_cum_log_prob = cum_log_prob
444         ↪ + log_prob
445
446         # 创建新序列
447         new_seq = torch.cat([seq, torch.

```

```

434     ↪ LongTensor([[token_id]]).to(device)], dim=1)
435
436     # 检查是否生成了EOS标记
437     is_eos = (token_id == eos_id)
438
439     candidates.append((
440     ↪ new_cum_log_prob, new_seq, is_eos))
441
442     # 按累积概率排序并保留前beam_size个候选项
443     candidates.sort(key=lambda x: x[0], reverse=
444     ↪ True)
445
446     # 更新beam队列
447     beam_queue = []
448     for cum_log_prob, seq, is_finished in
449     ↪ candidates[:beam_size]:
450         if is_finished:
451             # 如果序列已完成, 添加到
452             ↪ finished_beams
453             # 将长度惩罚应用到最终分数
454             length_penalty = ((5.0 + seq.size(1)
455             ↪ ) / 6.0) ** 0.65
456             finished_beams.append((cum_log_prob
457             ↪ / length_penalty, seq))
458         else:
459             # 否则继续扩展
460             beam_queue.append((cum_log_prob, seq
461             ↪ , is_finished))
462
463     # 如果所有beam都已完成或者达到最大长度的最后一
464     ↪ 步, 结束搜索
465     if len(beam_queue) == 0 or step == max_len -
466     ↪ 1:
467         # 将剩余未完成的序列添加到finished_beams
468         for cum_log_prob, seq, _ in beam_queue:
469             length_penalty = ((5.0 + seq.size(1)
470             ↪ ) / 6.0) ** 0.65
471             finished_beams.append((cum_log_prob
472             ↪ / length_penalty, seq))
473         break
474
475     # 选择分数最高的序列
476     if finished_beams:
477         finished_beams.sort(key=lambda x: x[0],
478         ↪ reverse=True)
479         best_seq = finished_beams[0][1]
480     else:
481         # 如果没有完成的序列, 选择当前最高分的序列
482         best_seq = beam_queue[0][1] if beam_queue
483         ↪ else torch.LongTensor([[eos_id]]).to(device)
484
485     batch_results.append(best_seq)
486
487     # 返回所有batch样本的最佳翻译
488     return batch_results
489
490 # 修改原有函数以使用Beam Search解码
491 def translate_and_evaluate_bleu(model, test_dataloader,
492     ↪ device, config, tgt_vocab):
493     model.eval()
494
495     # 创建ID到词的映射 (反向词表)

```

```

481     id_to_word = {v: k for k, v in tgt_vocab.items()}
482
483     # 存储所有生成的翻译和参考翻译
484     translations = []
485     references = []
486
487     # Beam Search参数
488     beam_size = 5 # 可以调整
489
490     for sample in tqdm.tqdm(test_dataloader):
491         src_input = sample["src_input"].to(device)
492         target = sample["target"] # 保留在CPU上用于提取
493         ↪ 参考翻译
494
495     # EOS标记ID
496     eos_id = 0
497
498     with torch.no_grad():
499         # 编码器前向传播 (只需要计算一次)
500         encoder_out = model.encoder(src_input)
501
502     # 使用Beam Search进行解码
503     decoded_seqs = beam_search_translate(
504         model,
505         src_input,
506         encoder_out,
507         device,
508         beam_size=beam_size,
509         max_len=100,
510         eos_id=eos_id
511     )
512
513     # 处理解码结果
514     for i, decoded in enumerate(decoded_seqs):
515         # 提取当前样本的生成序列 (跳过开始标记, 到
516         ↪ EOS标记或序列结束)
517         pred_ids = decoded[0, 1:].tolist() # 跳
518         ↪ 过第一个开始标记
519         if eos_id in pred_ids:
520             pred_ids = pred_ids[:pred_ids.index(
521             ↪ eos_id)]
522
523         # 将ID转换为词
524         pred_tokens = [id_to_word.get(idx, "<unk
525         ↪ >") for idx in pred_ids]
526         translations.append(" ".join(pred_tokens
527         ↪ ))
528
529     # 提取参考翻译
530     ref_ids = target[i].tolist()
531     if eos_id in ref_ids:
532         ref_ids = ref_ids[:ref_ids.index(
533         ↪ eos_id)]
534     ref_tokens = [id_to_word.get(idx, "<unk>
535     ↪ ") for idx in ref_ids]
536     references.append(" ".join(ref_tokens))
537
538     # 使用sacrebleu计算BLEU分数
539     bleu = sacrebleu.corpus_bleu(translations, [
540     ↪ references])
541     logger.info(f"Beam Search (beam_size={beam_size})
542     ↪ BLEU score: {bleu.score:.2f}")

```

```

533
534 # 保存翻译结果到文件
535 output_dir = os.path.dirname(config.save_dir)
536 with open(os.path.join(output_dir, "
    ↪ beam_search_translations.txt"), "w", encoding="
    ↪ utf-8") as f:
537     for trans in translations:
538         f.write(trans + "\n")
539 with open(os.path.join(output_dir, "references.txt")
    ↪ , "w", encoding="utf-8") as f:
540     for ref in references:
541         f.write(ref + "\n")
542
543 logger.info(f"Beam Search translations saved to {os.
    ↪ path.join(output_dir, 'beam_search_translations.
    ↪ txt'}}")
544
545 return bleu.score
546
547 def cli_main():
548     parser = argparse.ArgumentParser()
549     parser.add_argument("--config-path", type=str,
    ↪ required=True)
550     args = parser.parse_args()
551     train(args.config_path)
552
553
554 if __name__ == "__main__":
555     cli_main()

```

Code 6. train.py

train.sh 脚本本身非常简单, 通过调用 train.py 实现。

2.5.1. 词汇表加载和数据处理

```

1 def load_vocab(path):
2     with open(path, "r", encoding="utf-8") as f:
3         vocab = f.readlines()
4         vocab = [line.strip() for line in vocab]
5         return vocab
6
7 def load_data(corpus_path, vocab):
8     data = []
9     with open(corpus_path, "r", encoding="utf-8") as f:
10         for line in f:
11             line = line.strip()
12             if line:
13                 data.append(line)
14     return data

```

Code 7. 词汇表加载和数据处理

- load_vocab(path) 函数从指定路径加载一个 JSON 格式的词汇表文件。词汇表是一个字典, 键是词汇, 值是相应的索引。
- load_data(corpus_path, vocab) 函数从指定路径加载语料库文件。每一行代表一个
- 逐行读取文件内容, 去除每行的前后空白字符, 然后将行文本按空格拆分成单词 (或称为 tokens)。

- 对于每个单词, 通过查找词汇表字典获取对应的索引值。如果单词不在词汇表中, 使用索引 2 表示未知单词 (UNK)。
- 将每个句子的词汇索引转换为 PyTorch 的长整型张量 (torch.tensor)。
- 将转换后的张量添加到数据列表中。

2.5.2. dataset 定义

```

1 import torch
2 from torch.utils.data import Dataset
3
4 class MTDataset(Dataset):
5     def __init__(self, src_path, tgt_path, vocab):
6         self.src_path = src_path
7         self.tgt_path = tgt_path
8         self.vocab = vocab
9
10    def __getitem__(self, index):
11        src_line = self.src_path[index]
12        tgt_line = self.tgt_path[index]
13        src_tokens = self.vocab.encode(src_line)
14        tgt_tokens = self.vocab.encode(tgt_line)
15        return src_tokens, tgt_tokens
16
17    def __len__(self):
18        return len(self.src_path)

```

Code 8. dataset 定义

- 首先进行初始化, 加载源语言数据 self.src_data 和目标语言数据 self.tgt_data。确保两者长度相同。然后定义结束标记 self.eos_token。
- 通过返回字典的形式获取数据项。
- 最后返回数据集长度, 即 self.src_data 的长度。

句子或一段文本。

2.5.3. 批处理和损失函数

```

1 def padding_batch(values):
2     size = max(len(v) for v in values)
3     res = torch.zeros(len(values), size)
4     for i, v in enumerate(values):
5         res[i, :len(v)] = torch.tensor(v)
6     return res
7
8 def collate_fn(batch):
9     src_tokens = [item[0] for item in batch]
10    tgt_tokens = [item[1] for item in batch]
11    src_tokens = padding_batch(src_tokens)
12    tgt_tokens = padding_batch(tgt_tokens)
13    return src_tokens, tgt_tokens
14
15 def label_smoothed_nll_loss(pred, target, epsilon=0.1):
16     nll_loss = F.nll_loss(pred, target, ignore_index=0)
17     smooth_loss = -pred.sum(dim=-1, keepdim=True) *
    ↪ torch.log(pred + epsilon)
18     smooth_loss = smooth_loss.mean(dim=-1)
19     return nll_loss + smooth_loss

```

Code 9. 批处理和损失函数

- 首先计算所有序列的最大长度 size, 然后创建一个新的张量 res, 形状为 (batch_size, size), 并填充为 1, 再将每个序列复制到 res 的对应位置, 未使用的部分保留填充值 1。确保两者长度相同。然后定义结束标记 self.eos_token。
- 获取批次中所有样本的 src_input、tgt_input 和 target, 再对这些数据应用 padding_batch, 确保它们都有相同的长度, 接着返回包含填充后的 src_input、tgt_input 和 target 的字典。
- 如果 target 的维度比 lprobs 少一维, 给 target 添加一个维度, 然后计算负对数似然损失 nll_loss 和标签平滑损失 smooth_loss, 如果提供了 padding_idx, 将填充值的损失设为 0, 最后计算并返回总损失 loss, 以及未平滑的负对数似然损失 nll_loss。

句子或一段文本。

2.5.4. 设置随机种子

```
1 def setup_seed(seed:int):
2     torch.manual_seed(seed)
3     torch.cuda.manual_seed(seed)
4     torch.cuda.manual_seed_all(seed)
5     np.random.seed(seed)
6     random.seed(seed)
7     torch.backends.cudnn.deterministic = True
8     torch.backends.cudnn.benchmark = False
```

Code 10. 设置随机种子

分别设置 PyTorch CPU、单个 GPU、所有 GPU、Numpy 库和 python 标准库随机数种子。

2.5.5. 训练

```
1 import os
2 import os.path as osp
3 import sys
4 import tqdm
5 import json
6 import torch
7 import random
8 import logging
9 import argparse
10 import numpy as np
11 import torch.nn.functional as F
12 import sys
13 sys.path.append(osp.dirname(osp.dirname(osp.abspath(
14     ↪ __file__))))
15 from nmt.config import NMTConfig
16 from nmt.transformer import (
17     Encoder,
18     Decoder,
19     Transformer
20 )
21 # 添加sacrebleu导入
22 import sacrebleu
23 import heapq
24
25 logging.basicConfig(
```

```
26     format="%asctime)s | %(levelname)s | %(name)s | %(
27     ↪ message)s",
28     datefmt="%Y-%m-%d %H:%M:%S",
29     level=os.environ.get("LOGLEVEL", "INFO").upper(),
30     stream=sys.stdout,
31 )
32
33 logger = logging.getLogger("nmt.train")
34
35 def load_vocab(path):
36     with open(path, mode="r", encoding="utf-8") as fin:
37         vocab = json.load(fin)
38     return vocab
39
40
41 def load_data(corpus_path, vocab):
42     data = []
43
44     with open(corpus_path, mode="r", encoding="utf-8")
45     ↪ as fin:
46         for line in fin:
47             line = line.strip()
48
49             tokens = line.strip().split()
50
51             token_ids = []
52             for token in tokens:
53                 # 2 is UNK
54                 token_id = vocab.get(token, 2)
55                 token_ids.append(token_id)
56
57             token_ids = torch.tensor(token_ids, dtype=
58             ↪ torch.long)
59             data.append(token_ids)
60
61     return data
62
63 class MTDataset(torch.utils.data.Dataset):
64     def __init__(self, src_corpus_path, tgt_corpus_path,
65     ↪ src_vocab, tgt_vocab):
66         self.src_data = load_data(src_corpus_path,
67         ↪ src_vocab)
68         self.tgt_data = load_data(tgt_corpus_path,
69         ↪ tgt_vocab)
70
71         assert len(self.src_data) == len(self.tgt_data)
72         self.eos_token = torch.tensor([0])
73
74     def __getitem__(self, index):
75         src_input = self.src_data[index]
76
77         src_input = torch.cat([src_input, self.eos_token
78         ↪ ])
79
80         tgt_input = self.tgt_data[index]
81
82         target = torch.cat([tgt_input, self.eos_token])
83
84         tgt_input = torch.cat([self.eos_token, tgt_input
85         ↪ ])
```

```

80
81
82     return {
83         "src_input": src_input,
84         "tgt_input": tgt_input,
85         "target": target
86     }
87
88
89     def __len__(self):
90         return len(self.src_data)
91
92
93     def padding_batch(values):
94         size = max(v.shape[0] for v in values)
95
96         batch_size = len(values)
97
98         # 1 is padding
99         res = values[0].new(batch_size, size).fill_(1)
100
101         for i, v in enumerate(values):
102             res[i, : len(v)].copy_(v)
103
104         return res
105
106
107     def collate_fn(batch):
108         src_input = padding_batch(
109             [item["src_input"] for item in batch]
110         )
111         tgt_input = padding_batch(
112             [item["tgt_input"] for item in batch]
113         )
114         target = padding_batch(
115             [item["target"] for item in batch]
116         )
117         return {
118             "src_input": src_input,
119             "tgt_input": tgt_input,
120             "target": target
121         }
122
123
124     def label_smoothed_nll_loss(lprobs, target, epsilon,
125                                ↪ padding_idx=None):
126         if target.dim() == lprobs.dim() - 1:
127             target = target.unsqueeze(-1)
128
129         nll_loss = -lprobs.gather(dim=-1, index=target)
130         smooth_loss = -lprobs.sum(dim=-1, keepdim=True)
131
132         if padding_idx is not None:
133             pad_mask = target.eq(padding_idx)
134             nll_loss.masked_fill_(pad_mask, 0.0)
135             smooth_loss.masked_fill_(pad_mask, 0.0)
136         else:
137             nll_loss = nll_loss.squeeze(-1)
138             smooth_loss = smooth_loss.squeeze(-1)
139
140         nll_loss = nll_loss.sum()
141         smooth_loss = smooth_loss.sum()

```

```

141     eps_i = epsilon / (lprobs.size(-1) - 1)
142     loss = (1.0 - epsilon - eps_i) * nll_loss + eps_i *
143           ↪ smooth_loss
144     return loss, nll_loss
145
146     def setup_seed(seed: int):
147         torch.manual_seed(seed)
148         torch.cuda.manual_seed(seed)
149         torch.cuda.manual_seed_all(seed)
150         np.random.seed(seed)
151         random.seed(seed)
152
153
154     def train(config_path):
155         config = NMTConfig.load_config(config_path)
156
157         setup_seed(config.seed)
158
159         src_vocab = load_vocab(config.src_vocab_path)
160         tgt_vocab = load_vocab(config.tgt_vocab_path)
161
162         logger.info("Source language vocab size: {}".format(
163             ↪ len(src_vocab)))
164         logger.info("Target language vocab size: {}".format(
165             ↪ len(tgt_vocab)))
166
167         train_data = MTDataset(
168             config.src_train_corpus_path,
169             config.tgt_train_corpus_path,
170             src_vocab,
171             tgt_vocab
172         )
173
174         valid_data = MTDataset(
175             config.src_valid_corpus_path,
176             config.tgt_valid_corpus_path,
177             src_vocab,
178             tgt_vocab
179         )
180
181         test_data = MTDataset(
182             config.src_test_corpus_path,
183             config.tgt_test_corpus_path,
184             src_vocab,
185             tgt_vocab
186         )
187
188         encoder = Encoder(
189             d_model=config.d_model,
190             num_layers=config.num_encoder_layers,
191             num_heads=config.num_heads,
192             d_ff=config.d_ff,
193             dropout=config.dropout,
194             vocab_size=len(src_vocab)
195         )
196
197         decoder = Decoder(
198             d_model=config.d_model,
199             num_layers=config.num_decoder_layers,
200             num_heads=config.num_heads,
201             d_ff=config.d_ff,

```

```

200     dropout=config.dropout,
201     vocab_size=len(tgt_vocab)
202 )
203
204 model = Transformer(encoder, decoder)
205 model.train()
206
207 device = torch.device("cuda") if config.cuda else
208     ↪ torch.device("cpu")
209 model = model.to(device)
210
211 optimizer = torch.optim.Adam(
212     model.parameters(),
213     config.learning_rate,
214     betas=(config.adam_beta1, config.adam_beta2)
215 )
216
217 train_dataloader = torch.utils.data.DataLoader(
218     train_data,
219     config.batch_size,
220     shuffle=True,
221     collate_fn=collate_fn
222 )
223
224 valid_dataloader = torch.utils.data.DataLoader(
225     valid_data,
226     config.batch_size,
227     shuffle=False,
228     collate_fn=collate_fn
229 )
230
231 test_dataloader = torch.utils.data.DataLoader(
232     test_data,
233     config.batch_size,
234     shuffle=False,
235     collate_fn=collate_fn
236 )
237
238 '''
239 for sample in test_dataloader:
240     src_input = sample["src_input"]
241     tgt_input = sample["tgt_input"]
242     target = sample["target"]
243 '''
244
245 best_val_nll_loss = 999.00;
246 for i in tqdm.tqdm(range(config.epoch)):
247
248     for step, sample in enumerate(train_dataloader):
249
250         src_input = sample["src_input"]
251         tgt_input = sample["tgt_input"]
252         target = sample["target"]
253
254         src_input = src_input.to(device)
255         tgt_input = tgt_input.to(device)
256         target = target.to(device)
257
258         output = model(
259             src_input=src_input,
260             tgt_input=tgt_input,
261         )
262         lprobs = F.log_softmax(output, dim=-1)

```

```

261
262     lprobs = lprobs.view(-1, lprobs.shape[-1])
263     target = target.view(-1)
264
265     loss, nll_loss = label_smoothed_nll_loss(
266         ↪ lprobs, target, config.label_smoothing, 1)
267
268     num_tokens = target.ne(1).long().sum()
269
270     loss = loss / num_tokens
271     nll_loss = nll_loss / num_tokens
272
273     loss.backward()
274     optimizer.step()
275     optimizer.zero_grad()
276
277     # if (step + 1) % config.log_interval == 0:
278     #     logger.info(f"step: {step}, loss: {
279     ↪ loss.item(): .6f}, nll_loss: {nll_loss.item(): .6
280     ↪ f}")
281
282     # 每10轮在验证集上评估一次
283     if (i + 1) % 10 == 0 or i == 0:
284         valid_loss = 0.0
285         vali_nll_loss = 0.0
286         tokens_count = 0
287         for sample in valid_dataloader:
288
289             src_input = sample["src_input"]
290             tgt_input = sample["tgt_input"]
291             target = sample["target"]
292             src_input = src_input.to(device)
293             tgt_input = tgt_input.to(device)
294             target = target.to(device)
295
296             with torch.no_grad():
297                 output = model(
298                     src_input=src_input,
299                     tgt_input=tgt_input,
300                 )
301
302             lprobs = F.log_softmax(output, dim=-
303             ↪ 1)
304
305             lprobs = lprobs.view(-1, lprobs.
306             ↪ shape[-1])
307             target = target.view(-1)
308
309             loss, nll_loss =
310             ↪ label_smoothed_nll_loss(lprobs, target, config.
311             ↪ label_smoothing, 1)
312
313             num_tokens = target.ne(1).long().sum
314             ↪ ()
315
316             valid_loss += loss.item()
317             vali_nll_loss += nll_loss.item()
318             tokens_count += num_tokens
319
320         valid_loss /= tokens_count
321         vali_nll_loss /= tokens_count
322         logger.info(f"Epoch {i+1}/{config.epoch},

```

```

315     ↪ valid_loss: {valid_loss: .6f}, vali_nll_loss: {
316     ↪ vali_nll_loss: .6f}")
317
318     if (vali_nll_loss < best_val_nll_loss):
319         best_val_nll_loss = vali_nll_loss
320         logger.info(f"Epoch {i+1} SOTA in val,
321         ↪ performing nll as {best_val_nll_loss: .6f}")
322         test(test_dataloader, device, model, config
323         ↪ )
324         torch.save(model.state_dict(), os.path.
325         ↪ join(config.save_dir, f"best_checkpoint.pt"))
326
327         # 添加BLEU评估
328         translate_and_evaluate_bleu(model,
329         ↪ test_dataloader, device, config, tgt_vocab)
330     else:
331         logger.info(f"Epoch {i+1}/{config.epoch}
332         ↪ completed")
333
334 def test(test_dataloader, device, model, config):
335     test_loss = 0.0
336     test_nll_loss = 0.0
337     tokens_count = 0
338     for sample in test_dataloader:
339         src_input = sample["src_input"]
340         tgt_input = sample["tgt_input"]
341         target = sample["target"]
342
343         src_input = src_input.to(device)
344         tgt_input = tgt_input.to(device)
345         target = target.to(device)
346
347         with torch.no_grad():
348             output = model(
349                 src_input=src_input,
350                 tgt_input=tgt_input,
351             )
352
353             lprobs = F.log_softmax(output, dim=-1)
354
355             lprobs = lprobs.view(-1, lprobs.shape[-1])
356             target = target.view(-1)
357
358             loss, nll_loss = label_smoothed_nll_loss(
359                 ↪ lprobs, target, config.label_smoothing, 1)
360
361             num_tokens = target.ne(1).long().sum()
362
363             test_loss = test_loss + loss.item()
364             test_nll_loss = test_nll_loss + nll_loss.
365             ↪ item()
366             tokens_count = tokens_count + num_tokens
367
368     test_loss = test_loss / tokens_count
369     test_nll_loss = test_nll_loss / tokens_count
370     logger.info(f"test_loss: {test_loss: .6f},
371     ↪ test_nll_loss: {test_nll_loss: .6f}")
372
373 # 新增函数: Beam Search解码实现
374 def beam_search_translate(model, src_input, encoder_out,
375     ↪ device, beam_size=5, max_len=100, eos_id=0):

```

```

376     """
377     使用Beam Search算法进行解码翻译
378
379     Args:
380         model: Transformer模型
381         src_input: 源语言输入
382         encoder_out: 编码器输出
383         device: 计算设备
384         beam_size: beam大小
385         max_len: 最大生成长度
386         eos_id: 结束标记ID
387
388     Returns:
389         最终的翻译序列
390     """
391     batch_size = src_input.size(0)
392     vocab_size = model.decoder.linear.out_features
393
394     # 存储每个batch样本的beam结果
395     batch_results = []
396
397     # 对batch中的每个样本单独处理
398     for batch_idx in range(batch_size):
399         # 提取当前样本的编码器输出和padding mask
400         single_encoder_out = encoder_out[0][:, batch_idx
401         ↪ :batch_idx+1]
402         single_encoder_padding_mask = encoder_out[1][
403         ↪ batch_idx:batch_idx+1]
404
405         # 初始化队列, 每个元素为(累积log概率, 当前序列, 已
406         ↪ 完成标记)
407         beam_queue = [(0.0, torch.LongTensor([[eos_id]]),
408         ↪ .to(device), False)]
409
410         # 已完成的序列
411         finished_beams = []
412
413         # beam search主循环
414         for step in range(max_len):
415             # 临时存储当前步骤的候选项
416             candidates = []
417
418             # 处理所有当前beam中的候选项
419             for cum_log_prob, seq, is_finished in
420                 ↪ beam_queue:
421                 # 如果序列已完成, 将其添加到候选项, 不再扩
422                 ↪ 展
423                 if is_finished:
424                     candidates.append((cum_log_prob, seq
425                 ↪ , is_finished))
426                 continue
427
428             # 用于当前序列的解码器输入
429             decoder_input = seq
430
431             # 解码器前向传播
432             with torch.no_grad():
433                 decoder_out = model.decoder(
434                 ↪ decoder_input, single_encoder_out,
435                 ↪ single_encoder_padding_mask)
436
437             # 获取最后位置的log概率

```

```

419         log_probs = F.log_softmax(
420             ↪ decoder_out[:, -1, :], dim=-1)
421
422         # 获取前beam_size个最大概率的词
423         topk_log_probs, topk_indices =
424         ↪ log_probs.topk(beam_size)
425
426         # 将新候选项添加到列表
427         for i in range(beam_size):
428             token_id = topk_indices[0, i].
429             ↪ item()
430             log_prob = topk_log_probs[0, i].
431             ↪ item()
432
433             # 计算新的累积概率
434             new_cum_log_prob = cum_log_prob
435             ↪ + log_prob
436
437             # 创建新序列
438             new_seq = torch.cat([seq, torch.
439             ↪ LongTensor([[token_id]]).to(device)], dim=1)
440
441             # 检查是否生成了EOS标记
442             is_eos = (token_id == eos_id)
443
444             candidates.append((
445             ↪ new_cum_log_prob, new_seq, is_eos))
446
447             # 按累积概率排序并保留前beam_size个候选项
448             candidates.sort(key=lambda x: x[0], reverse=
449             ↪ True)
450
451             # 更新beam队列
452             beam_queue = []
453             for cum_log_prob, seq, is_finished in
454             ↪ candidates[:beam_size]:
455                 if is_finished:
456                     # 如果序列已完成, 添加到
457                     ↪ finished_beams
458                     # 将长度惩罚应用到最终分数
459                     length_penalty = ((5.0 + seq.size(1)
460                     ↪ ) / 6.0) ** 0.65
461                     finished_beams.append((cum_log_prob
462                     ↪ / length_penalty, seq))
463                 else:
464                     # 否则继续扩展
465                     beam_queue.append((cum_log_prob, seq
466                     ↪ , is_finished))
467
468             # 如果所有beam都已完成或者达到最大长度的最后一
469             ↪ 步, 结束搜索
470             if len(beam_queue) == 0 or step == max_len -
471             ↪ 1:
472                 # 将剩余未完成的序列添加到finished_beams
473                 for cum_log_prob, seq, _ in beam_queue:
474                     length_penalty = ((5.0 + seq.size(1)
475                     ↪ ) / 6.0) ** 0.65
476                     finished_beams.append((cum_log_prob
477                     ↪ / length_penalty, seq))
478                 break
479
480             # 选择分数最高的序列

```

```

464         if finished_beams:
465             finished_beams.sort(key=lambda x: x[0],
466             ↪ reverse=True)
467             best_seq = finished_beams[0][1]
468         else:
469             # 如果没有完成的序列, 选择当前最高分的序列
470             best_seq = beam_queue[0][1] if beam_queue
471             ↪ else torch.LongTensor([[eos_id]]).to(device)
472
473             batch_results.append(best_seq)
474
475             # 返回所有batch样本的最佳翻译
476             return batch_results
477
478             # 修改原有函数以使用Beam Search解码
479             def translate_and_evaluate_bleu(model, test_dataloader,
480             ↪ device, config, tgt_vocab):
481                 model.eval()
482
483                 # 创建ID到词的映射 (反向词表)
484                 id_to_word = {v: k for k, v in tgt_vocab.items()}
485
486                 # 存储所有生成的翻译和参考翻译
487                 translations = []
488                 references = []
489
490                 # Beam Search参数
491                 beam_size = 5 # 可以调整
492
493                 for sample in tqdm.tqdm(test_dataloader):
494                     src_input = sample["src_input"].to(device)
495                     target = sample["target"] # 保留在CPU上用于提取
496                     ↪ 参考翻译
497
498                     # EOS标记ID
499                     eos_id = 0
500
501                     with torch.no_grad():
502                         # 编码器前向传播 (只需要计算一次)
503                         encoder_out = model.encoder(src_input)
504
505                         # 使用Beam Search进行解码
506                         decoded_seqs = beam_search_translate(
507                             model,
508                             src_input,
509                             encoder_out,
510                             device,
511                             beam_size=beam_size,
512                             max_len=100,
513                             eos_id=eos_id
514                         )
515
516                     # 处理解码结果
517                     for i, decoded in enumerate(decoded_seqs):
518                         # 提取当前样本的生成序列 (跳过开始标记, 到
519                         ↪ EOS标记或序列结束)
520                         pred_ids = decoded[0, 1:].tolist() # 跳
521                         ↪ 过第一个开始标记
522                         if eos_id in pred_ids:
523                             pred_ids = pred_ids[:pred_ids.index(
524                             ↪ eos_id)]

```

Code 11. 训练

- 函数加载目标语言的词汇表。

- 实例化 `MTDataset` 类，传入源语言和目标语言的训练语料路径以及对应的词汇表和验证语料路径及词汇表。
- 使用配置文件中的参数，实例化 `Encoder` 类。参数包括模型维度、层数、头数、前馈网络维度、dropout 概率以及词汇表大小。同样，使用配置文件中的参数，实例化 `Decoder` 类。然后使用编码器和解码器实例，创建 `Transformer` 模型，并将其设置为训练模式 `model.train()`。

3.1. sentencepiece

```

$ pip install autogen
Collecting autogen
  Downloading autogen-0.1.10-py3-none-any.whl (30 kB)
Collecting autogen
  Downloading autogen-0.1.10-py3-none-any.whl (44 kB)
    44 kB 561 kB/s
Collecting twine-5.1.0-py3-none-any.whl (38 kB)
  Downloading twine-5.1.0-py3-none-any.whl (38 kB)
Requirement already satisfied: PyYAML in /usr/lib/python3/dist-packages (from autogen) (5.3.1)
Requirement already satisfied: setuptools in /usr/lib/python3/dist-packages (from autogen) (45.2.0)
Collecting dogstest
  Downloading dogstest-0.2.1-rc.0-py3-none-any.whl (25 kB)
Requirement already satisfied: tomli; python_version < "7.11" in /usr/local/lib/python3.8/dist-packages (from autogen->autogen) (2.0.1)
Collecting pygtrie-0.2.10
  Downloading pygtrie-0.2.10-py3-none-any.whl (31 kB)
Collecting urllib3-1.26.0
  Downloading urllib3-1.26.0-py3-none-any.whl (121 kB)
    121 kB 624 kB/s
Collecting rfc9398-2.0-py2.py3-none-any.whl (31 kB)
  Downloading rfc9398-2.0-py2.py3-none-any.whl (31 kB)
Requirement already satisfied: korymb-15.1 in /usr/lib/python3/dist-packages (from twine->autogen) (15.0.1)
Collecting importlib_metadata-6.8
  Downloading importlib_metadata-6.8-py3-none-any.whl (34 kB)
Collecting readme_renderer-39.0
  Downloading readme_renderer-39.0-py3-none-any.whl (13 kB)
Collecting paginate-1.10.0-py3-none-any.whl (30 kB)
  Downloading paginate-1.10.0-py3-none-any.whl (30 kB)
Collecting requests-toolbelt-1.0.0-py3-none-any.whl (54 kB)
  Downloading requests_toolbelt-1.0.0-py3-none-any.whl (54 kB)
    54 kB 5.8 MB/s
Requirement already satisfied: urllib3-1.26.0 in /usr/local/lib/python3.8/dist-packages (from twine->autogen) (1.26.0)
Requirement already satisfied: requests-2.20 in /usr/local/lib/python3.8/dist-packages (from twine->autogen) (2.31.0)

```

Figure 3. 安装 sentencepiece 必要的依赖

```
> cd /mnt/d/desktop/homework/2023-2024/mnt/_experiment/mnt/_experiment/antennecisee/python3/setup.py build_wheel
Running build_wheel
Running building
Creating build/lib.linux-x86_64-3.8
Copying file antennecisee/_init_.py -> build/lib.linux-x86_64-3.8/antennecisee
Copying file antennecisee/_version.py -> build/lib.linux-x86_64-3.8/antennecisee
Copying file antennecisee/antennecisee_model_pb2.py -> build/lib.linux-x86_64-3.8/antennecisee
Copying file antennecisee/antennecisee_model_pb2.py -> build/lib.linux-x86_64-3.8/antennecisee
Running build_ext
I:/usr/local/lib\site-packages\setuptools\command\build_ext.py:179: UserWarning: The directory containing protobin-lite.pc
is not found in the pkg-config search path.
Perhaps you should add the directory containing 'protobin-lite.pc'
to the PKG_CONFIG_PATH environment variable.
Package 'protobin-lite', required by 'antennecisee', not found
Unable to find protoc-gen-mypy config
C:\Users\user> cd /mnt/d/desktop/homework/2023-2024/mnt/_experiment/mnt/_experiment/antennecisee/python3/cd .
Building wheel
WARNING: Can't create directory 'build': File exists
C:\Users\user> cd /mnt/d/desktop/homework/2023-2024/mnt/_experiment/mnt/_experiment/antennecisee/cd build
File root
ERROR: 9.2.1
Found CMakeTool:/usr/lib/x86_64-linux-gnu/libcmtool.minimal.so
Configuring done
Generating done
The files have been written to: /mnt/d/desktop/homework/2023-2024/mnt/_experiment/mnt/_experiment/antennecisee/build
```

Figure 4. 安装 sentencepiece 的 c++ engine

安装 BLEU

```
New service...
root@kali:~/Desktop# ./msf4/desktop/homework0203-2024/mst_expert/mst_expert@microsoft sacrebule -t wat1/system -l zh-cn --echo Niufranc | sacrebule -t wat1/
sacrebule: [Unknown test set "wat1/system"]
sacrebule: Please run with --list to see the available test sets.
sacrebule: System and reference streams have different lengths.
sacrebule: This could be an issue with your system output or with sacrebule's reference database if -t is given.
sacrebule: For the latter, try cleaning out the cache by typing:
sacrebule: rm -r /root/.sacrebule/wat1/system

sacrebule: The test set will be re-downloaded the next time you run sacrebule.
root@kali:~/Desktop# ./msf4/desktop/homework0203-2024/mst_expert/mst_expert@microsoft sacrebule -t wat17 -e nde -o output.detok.txt
[Feedback: most recent call last]:
File "/usr/local/bin/sacrebule", line 8, in <module>
    sys.exit(main())
File "/usr/local/bin/python3.8/dist-packages/sacrebule/sacrebule.py", line 419, in main
    for line in smart_open(filename, encoding=args.encoding):
File "/usr/local/bin/python3.8/dist-packages/sacrebule/sacrebule.py", line 251, in smart_open
    return open(file, mode=mode, encoding=encoding, newline='').readlines()
FileNotFoundError: [Errno 2] No such file or directory: 'output.detok.txt'
root@kali:~/Desktop# ./msf4/desktop/homework0203-2024/mst_expert/mst_expert@microsoft sacrebule cat output.detok.txt | sacrebule -t wat17 -e nde
cat: output.detok.txt: No such file or directory
sacrebule: System and reference streams have different lengths.
sacrebule: This could be an issue with your system output or with sacrebule's reference database if -t is given.
sacrebule: For the latter, try cleaning out the cache by typing:
sacrebule: rm -r /root/.sacrebule/wat17

sacrebule: The test sets will be re-downloaded the next time you run sacrebule.
root@kali:~/Desktop# ./msf4/desktop/homework0203-2024/mst_expert/mst_expert@microsoft sacrebule ref1 ref2 -t system --blue chief ter
```

Figure 5. 安装 BLEU

验证 BLEU 成功安装

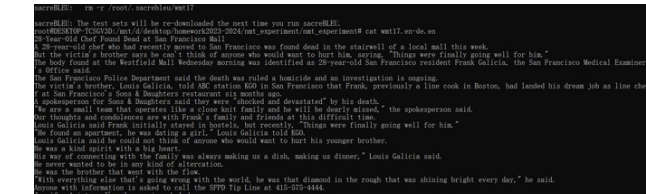


Figure 6. 验证 BLEU 成功安装

3.3. 训练

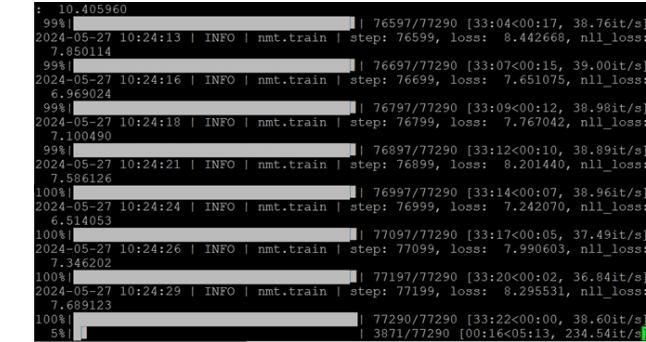


Figure 7. 训练

训练会找出来一个最好的 checkpoint, 之后遍历每一种 beam size 的结果, 当 beam size 等于 1 的时候是贪心算法。

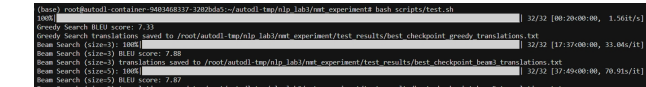


Figure 8. bleu score

4. 心得

这次的实验实践上来讲是最难的一次, 一开始预处理数据集开始就需要自己动手实现, 到 sentenpiece 的时候不仅需要 pip install 还需要把 c++ 的 engine 装上, 解码算法很重要, beam size 一开始往上增加的时候会明显增加 bleu 的分数, detokenize 也会明显增加 bleu 的分数, 但是这个分数不会到很高, 因为用的是很基本的 transformer, 加上英译中本身就比英译德要难得多, 这个分数和助教讨论过了。后续改进 transformer 的话能大幅提升 bleu 的分数。